Максимальный поток, минимальный разрез, паросочетания

Содержание

01 **Паросочетания**
Распределение объектов по парам без конфликтов

02 **Максимальный поток**
Поиск максимальной пропускной способности сети

03 **Минимальный разрез**
Минимальное «узкое место» сети

04 **Потоки и паросочетания**
Сведение одной задачи к другой

01. Паросочетания

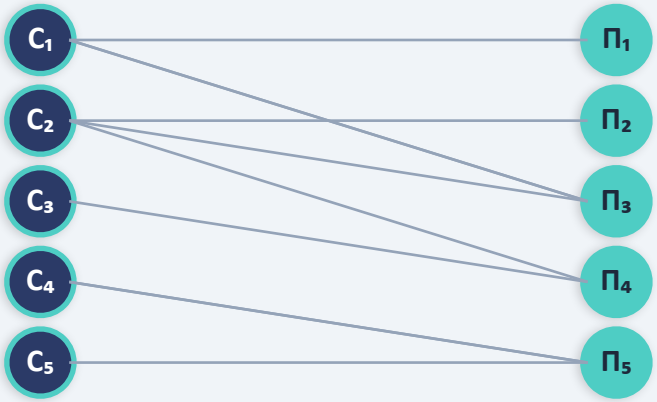
Задача распределения без конфликтов

Зачем нужны паросочетания?

Задача

В университете 5 студентов и 5 дипломных проектов. Каждый студент выбрал 2–3 проекта, которые ему интересны. Как распределить проекты так, чтобы максимум студентов получили интересный им проект — и никакой проект не достался двоим?

Модель:



Студенты
= вершины левой доли

Проекты
= вершины правой доли

Ребро
= «студент хочет проект»

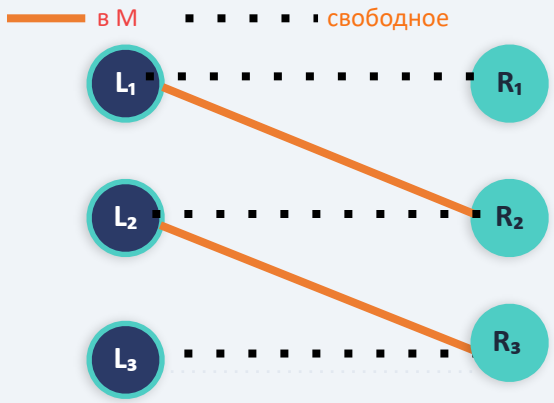
Паросочетание
= назначение без конфликтов

Определение паросочетания

Паросочетание (matching) *M* — множество рёбер графа такое, что никакие два ребра не имеют общей вершины.

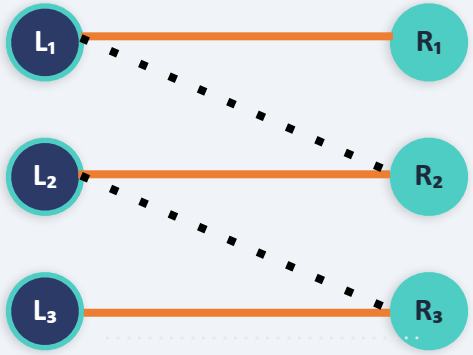
Максимальное паросочетание — паросочетание с наибольшим числом рёбер.

Паросочетание |M| = 2. Как улучшить?



«развернём» рёбра на пути

После разворота: |M| = 3



Свободные рёбра и рёбра из M чередуются!

Чередующаяся и увеличивающая цепь

Чередующаяся цепь — путь, в котором рёбра из M (паросочетания) и не из M чередуются.
Увеличивающая цепь — чередующаяся цепь, которая начинается и заканчивается свободной вершиной.

Пример: 1 — 8 — 4 — 6 — 3 — 7

Если нашли увеличивающую цепь — можно «развернуть» рёбра и увеличить паросочетание на 1.

Теорема Бержа

Теорема: Паросочетание M максимально тогда и только тогда, когда не существует увеличивающей цепи относительно M.

Идея доказательства:

$\Longrightarrow$ **(если M максимально → нет увелич. цепи)**

Если бы была увеличивающая цепь, мы бы развернули рёбра и получили M' с |M'| = |M| + 1. Противоречие с максимальностью M.

$\Longleftarrow$ **(если нет увелич. цепи → M максимально)**

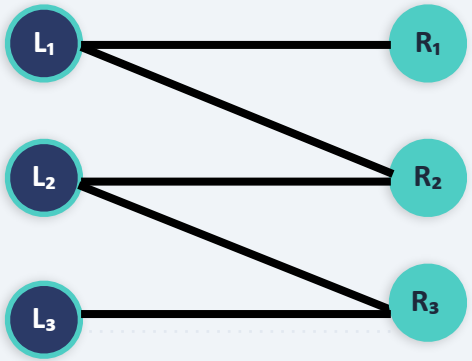
Пусть M' — истинное максимальное паросочетание, |M'| > |M|. Рассмотрим симметрическую разность M ⊕ M'. Она состоит из чередующихся цепей и циклов. В циклах |M| = |M'|, значит есть цепь, где рёбер M' больше — это увеличивающая цепь. Противоречие.

Алгоритм Куна (двудольный граф)

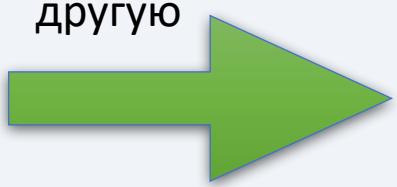
Идея: Для каждой свободной вершины левой доли пытаемся найти увеличивающую цепь с помощью DFS. Если нашли — разворачиваем рёбра.

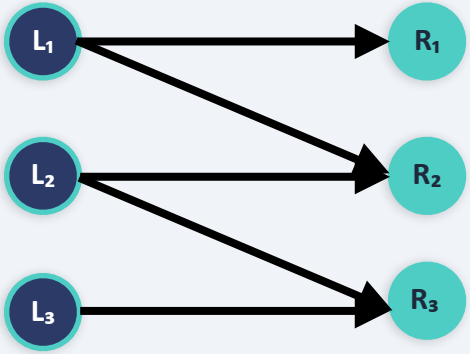
1. Изначально паросочетание пустое. Ориентируем все рёбра из левой доли в правую.
2. Берём свободную вершину из левой доли. DFS ищет путь до свободной вершины правой доли.
3. Если путь найден — разворачиваем все рёбра на пути (свободные → в M, из M → свободные).
4. Повторяем для каждой вершины левой доли. Когда все обработаны — M максимально.

Алгоритм Куна: пример

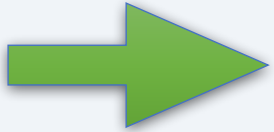


Ориентируем рёбра из 1 доли в другую

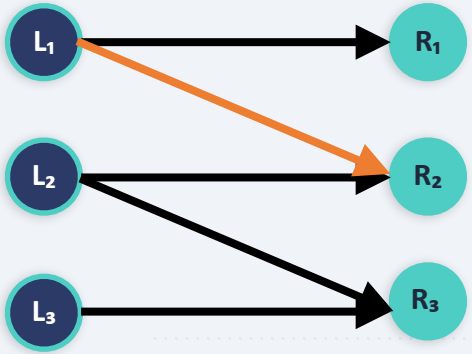


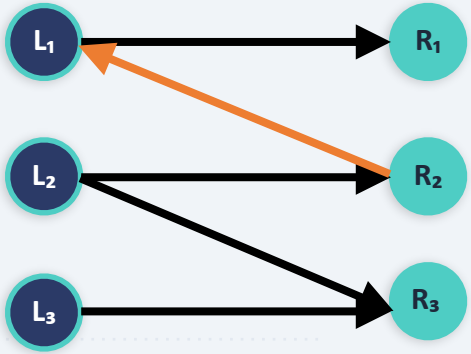


Запускаем DFS из L1
Пускай он пошёл в R2

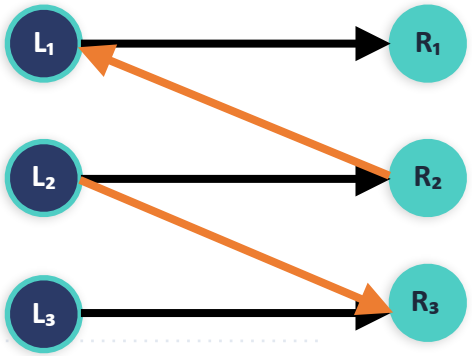


Из R2 мы никуда идти не можем -> останавливаемся -> инвертируем ребро (L1, R1) т.е добавляем его в паросочетание

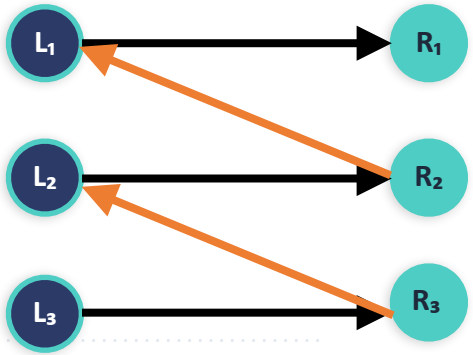




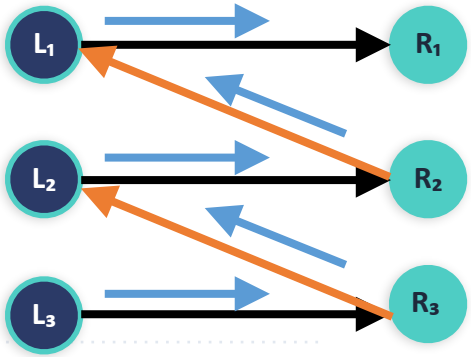
Запускаем DFS из L2. Пускай он пошёл в R3



Аналогично инвертируем ребро (L2, R3), добавляя его в M

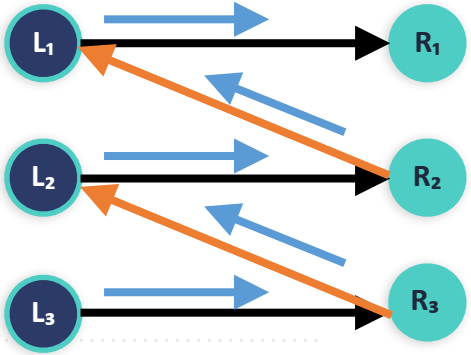


Запускаем DFS из L3, он может пойти только в R3 -> DFS не останавливается тк можем пойти в L2 по обратному ребру -> по прямому ребру идем в R2 -> по обратному ребру в L1 -> по прямому ребру в R1



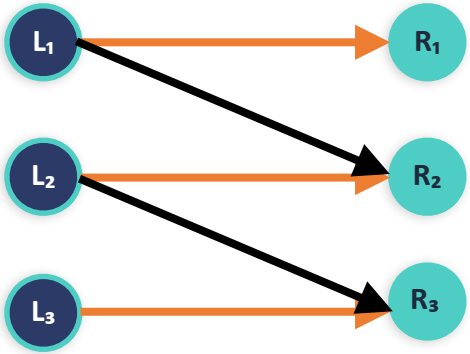
Итого мы запустили DFS из левой доли и остановились в правой

Итого мы запустили DFS из левой доли и остановились в правой



Нашли увеличивающую цепь

Инвертируем рёбра



Асимптотика: O(V · E) — для каждой из V вершин запускаем DFS за O(E)

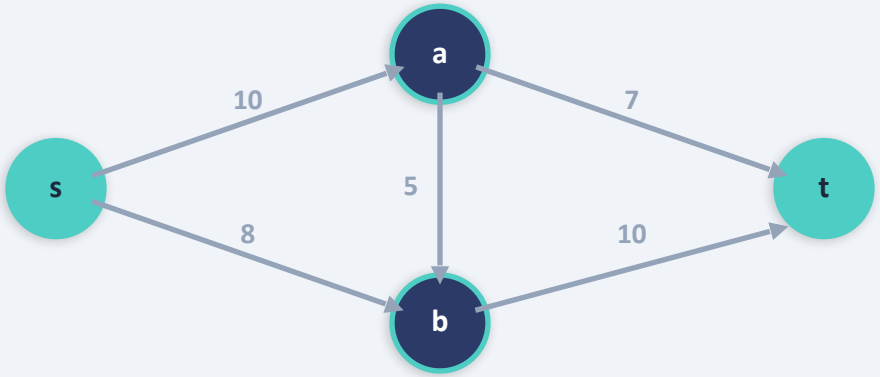
02. Максимальный поток

Сколько можно пропустить через сеть?

Зачем нужен максимальный поток?

Задача

Нефтепровод состоит из труб разного диаметра (= пропускная способность). Нефть идёт от месторождения (источник s) до завода (сток t). Какой максимальный объём нефти в сутки можно прокачать по этой сети?



Ориентированный граф:
вершины = узлы трубопровода
рёбра = трубы
веса = пропускная способность

Ответ: максимальный поток = 17

Определения: сеть и поток

Сеть — ориентированный граф $G = (V, E)$ с двумя выделенными вершинами: источник s и сток t, и функцией пропускной способности $c(u, v) \geq 0$.

Поток — функция $f(u, v)$, удовлетворяющая трём свойствам:

Ограничение

$f(u,v) \leq c(u,v)$
Поток не превышает пропускную способность

Антисимметрия

$f(u,v) = -f(v,u)$
Поток в одну сторону = минус поток в другую

Сохранение

$\Sigma f(u,v) = 0$ для всех $v \neq s, t$
Что втекает — то вытекает

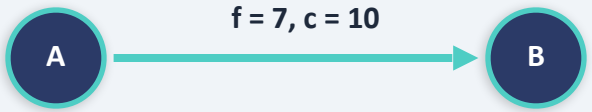
Величина потока $|f| = \Sigma f(s, v)$ — суммарный поток из источника

Остаточная сеть

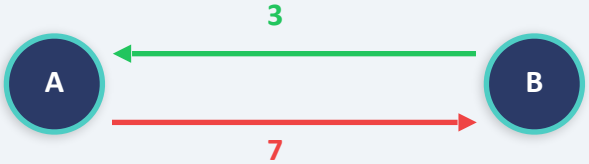
Для каждого ребра (u, v) с пропускной способностью c и потоком f:

Прямое ребро (u → v): остаточная пропускная способность = c(u,v) − f(u,v)
Обратное ребро (v → u): остаточная пропускная способность = f(u,v)

Исходное ребро:



Остаточная сеть:



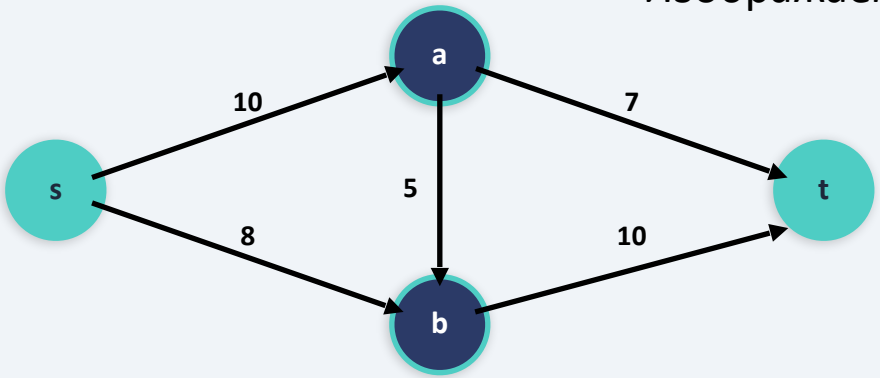
Альтернативный взгляд (два массива):
cap[A][B] = 3 (сколько ещё можно), back[B][A] = 7 (сколько можно отменить)
Инвариант: cap[u][v] + back[v][u] = c(u,v) всегда

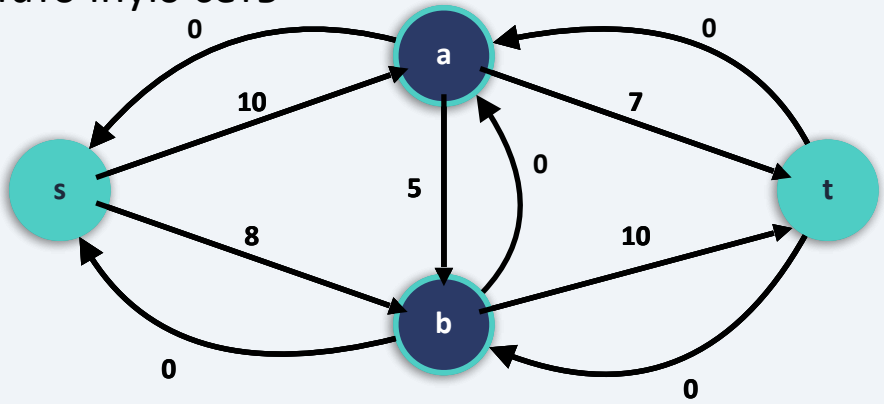
Алгоритм Форда-Фалкерсона

1. Инициализация: поток по всем рёбрам = 0. Остаточная сеть = исходный граф.
2. Находим любой путь из s в t в остаточной сети (DFS).
3. Находим bottleneck — минимальную остаточную пропускную способность на пути.
4. Проталкиваем поток: для каждого ребра на пути вычитаем bottleneck из прямого, прибавляем к обратному.
5. Повторяем шаги 2–4, пока существует путь. Когда пути нет — поток максимален.

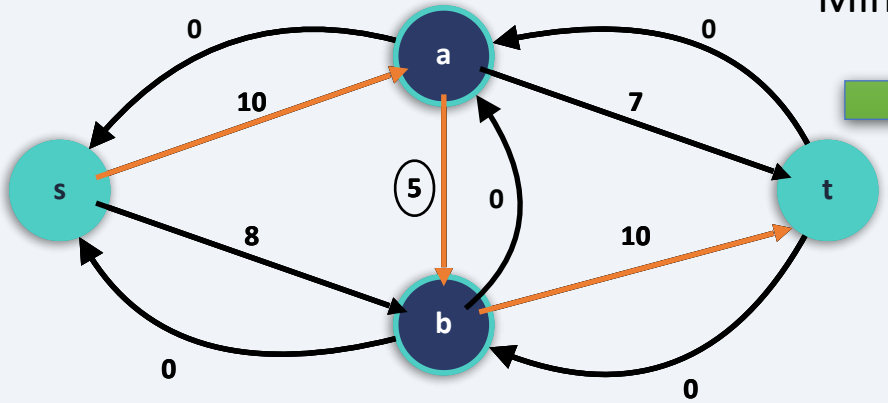
Форд-Фалкерсон: пример

Изображаем остаточную сеть

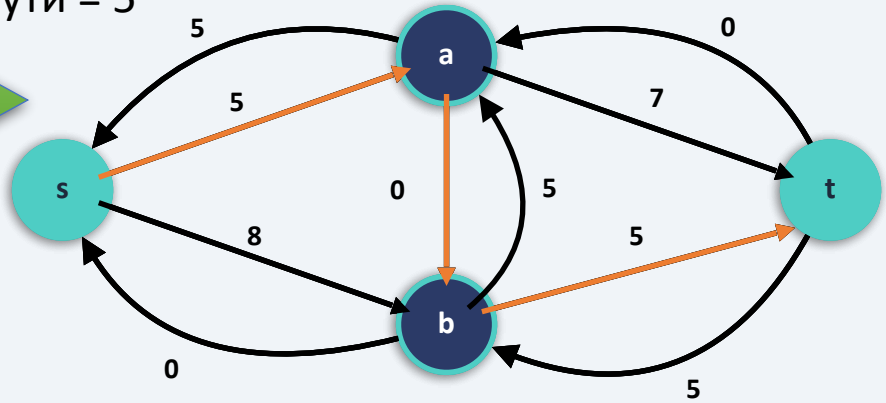




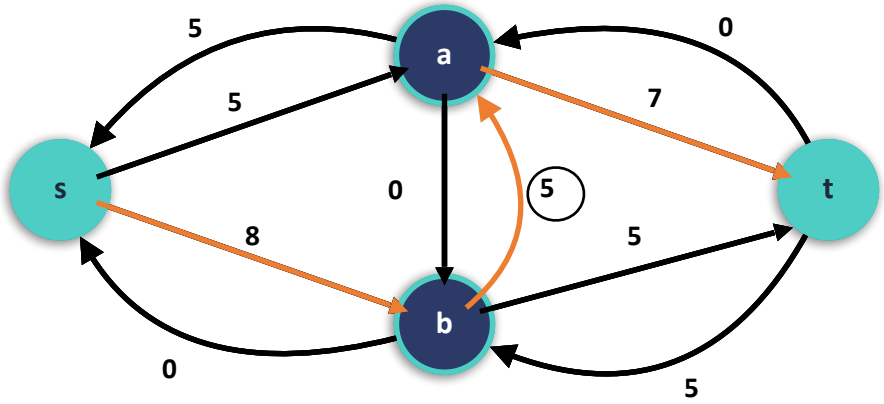
Запускаем DFS из s в t



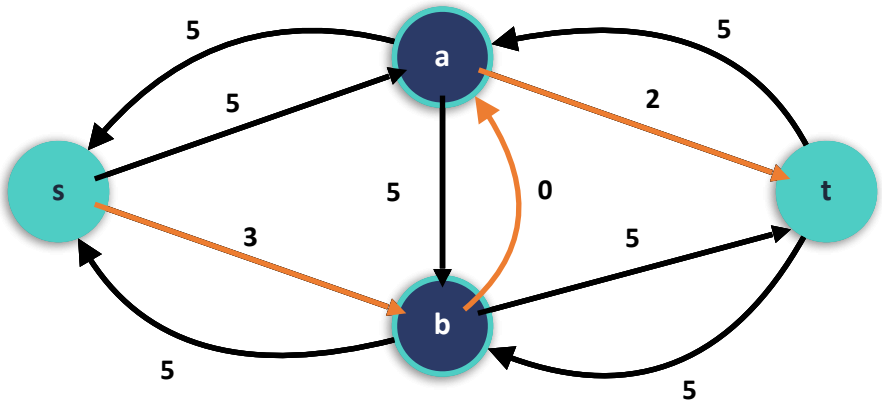
Min на пути = 5

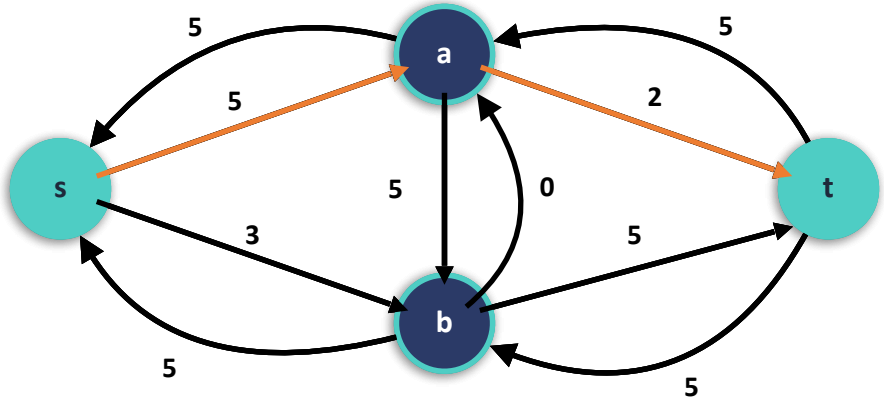


Опять ищем путь с помощью DFS

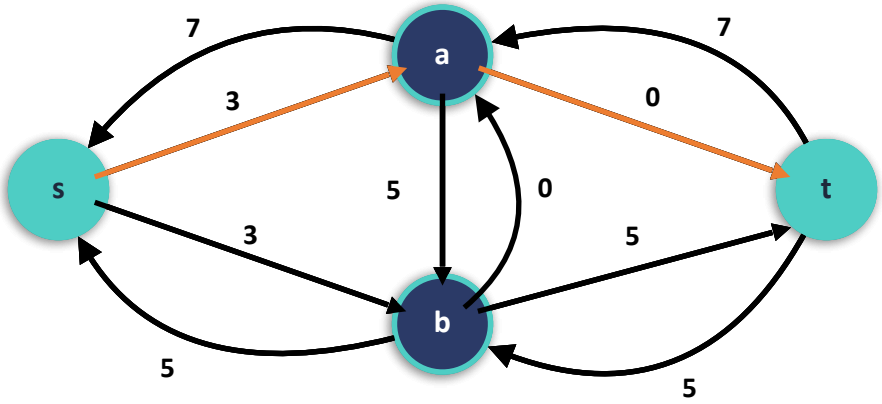


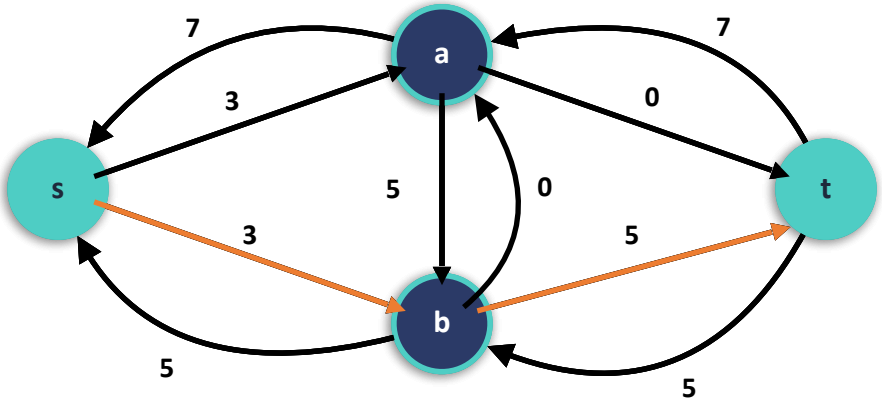
Min = 5



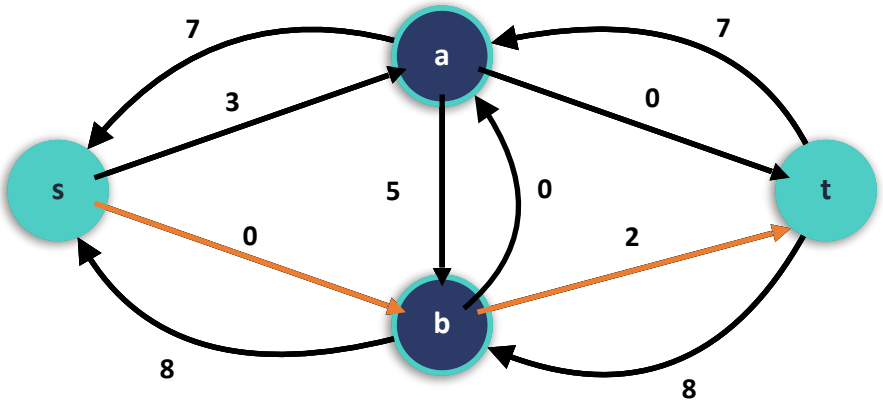


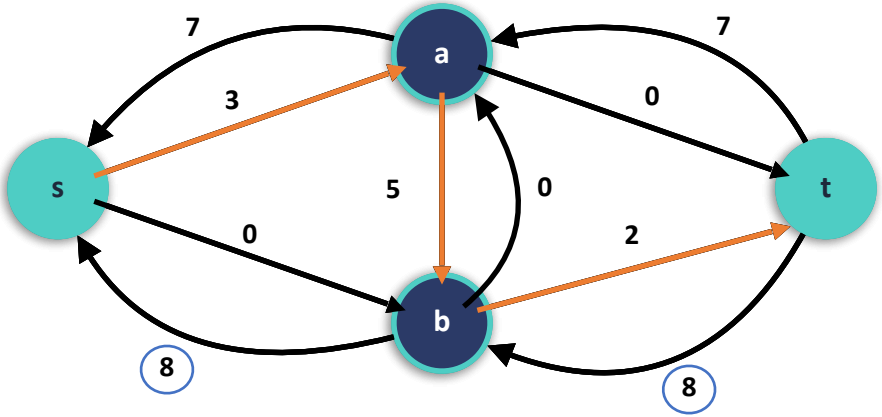
Min = 2



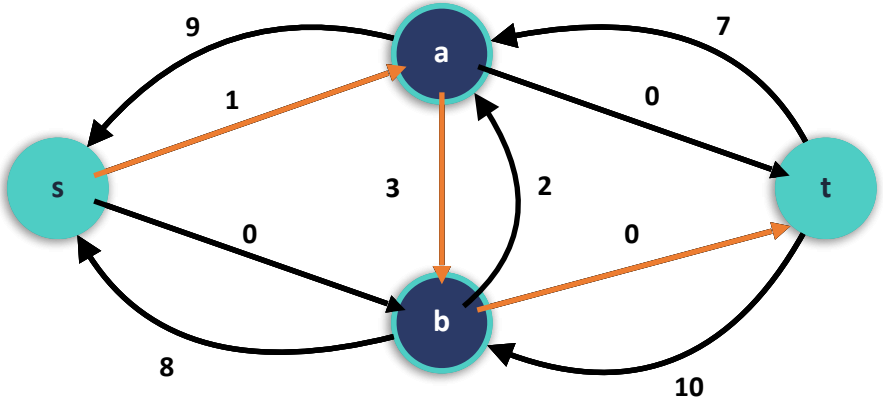

a
s
t
b
7
3
7
0
5
0
3
5
5
5

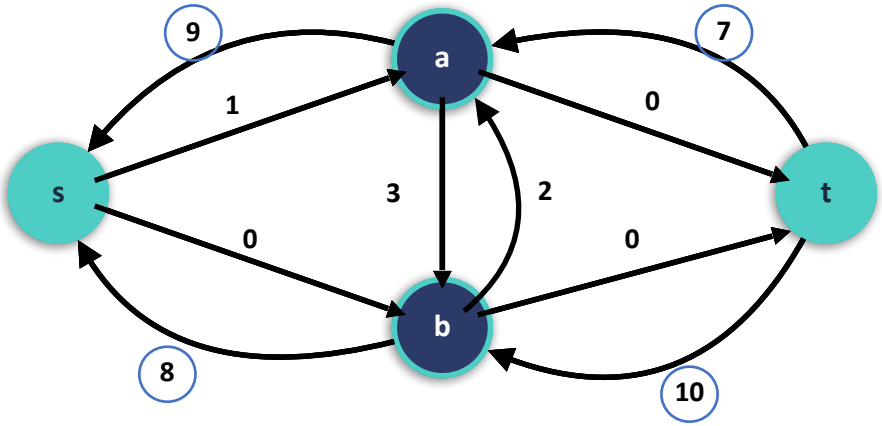
Min = 3


a
s
t
b
7
3
7
0
5
0
0
2
8
8


a
s
t
b
7
3
7
0
5
0
0
2
8
8

Min = 2

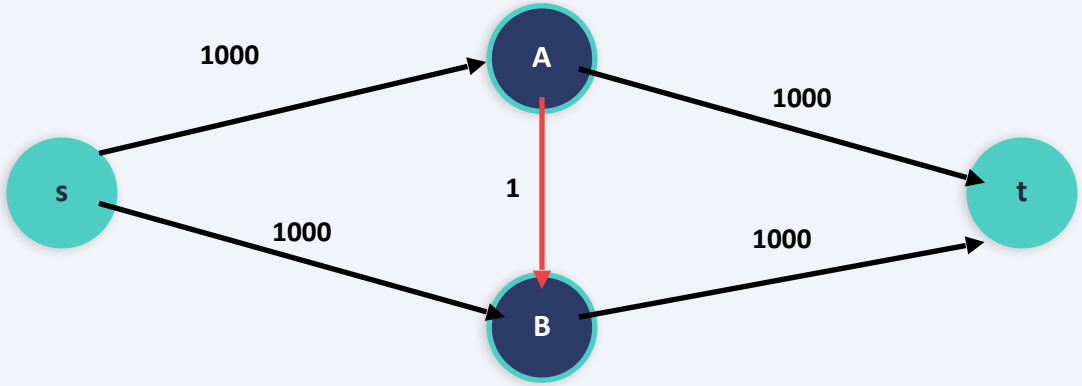

a
s
t
b
9
1
7
0
3
2
0
0
8
10



Пути из s в t больше нет. Ответ у нас в остаточной сети -> сумма рёбер выходящих из стока = сумме рёбер входящих в исток

Проблема DFS: пример-убийца

Граф, на котором DFS делает 2000 итераций вместо 2:



DFS может чередовать:

s → A → B → t (поток +1)

s → B → A → t (поток +1)

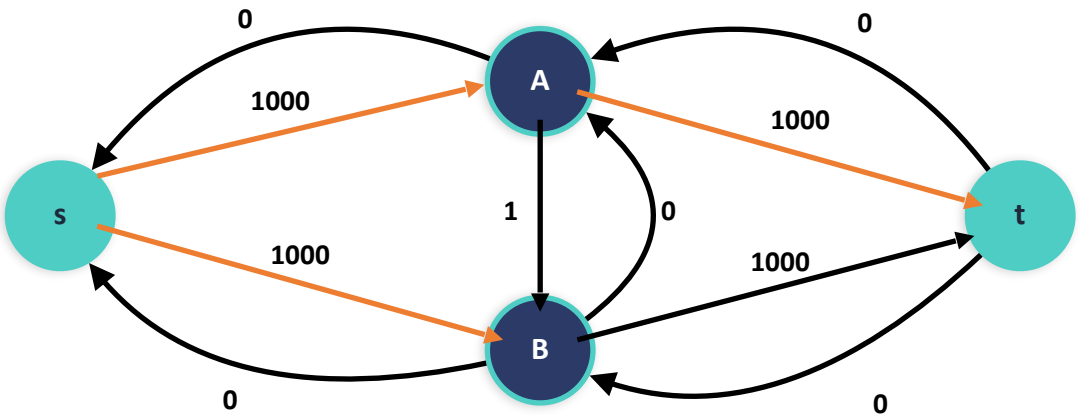
...и так 2000 раз!

BFS сразу находит:

s → A → t (поток +1000)

s → B → t (поток +1000)

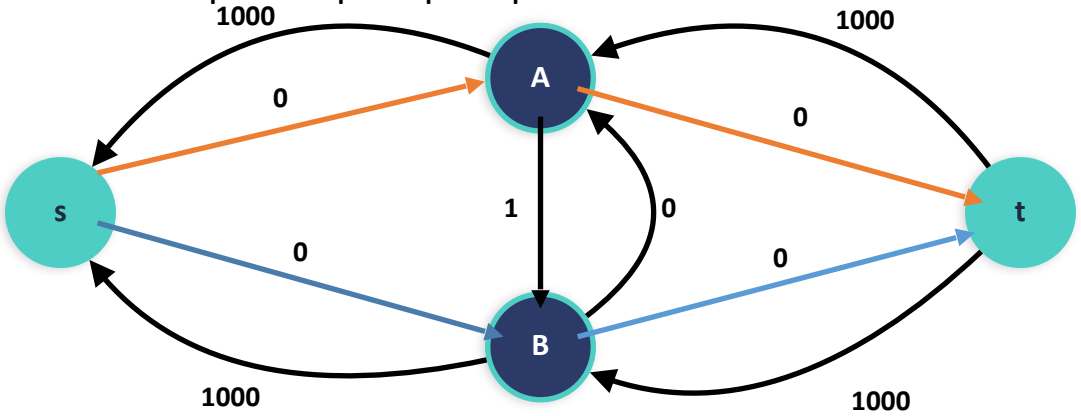
Всего 2 итерации!



Запустим BFS

BFS найдёт кратчайший путь по количеству ребер (s->A->t)

По массиву предков восстанавливаем маршрут и также вычитаем величину потока из прямых рёбер и прибавляем к остаточной сети



Аналогично BFS найдет путь s->B->t , после чего завершиться

Алгоритм Эдмондса-Карпа

Ключевая идея: Форд-Фалкерсон + BFS. Всегда выбираем кратчайший (по числу рёбер) увеличивающий путь.

Почему BFS решает проблему:

Расстояния не уменьшаются

d(v) — длина кратчайшего пути от s до v. BFS гарантирует: d(v) от итерации к итерации может только расти или оставаться.

Воскрешение стоит дорого

Если ребро «умерло» (стало bottleneck), чтобы оно «воскресло», d(u) должно вырасти минимум на 2. Одно ребро умирает не более O(V) раз.

DFS этого не даёт

DFS не гарантирует рост расстояний → одно ребро может умирать и воскресать неограниченно.

Эдмондс-Карп: пример (BFS выбирает кратчайший путь)

Сравним DFS и BFS на графе: s→A (100), s→B (100), A→B (1), A→t (100), B→t (100)

BFS (Эдмондс-Карп)

1. **s → A → t**
   длина 2 · bottleneck = 100 · поток = 100

2. **s → B → t**
   длина 2 · bottleneck = 100 · поток = 200

2 итерации, max flow = 200

DFS (Форд-Фалкерсон)

1. s → A → B → t (длина 3, bn=1)
2. s → B → A → t (длина 3, bn=1)
3. s → A → B → t (длина 3, bn=1)

пинг-понг через ребро A↔B

200 итераций, тот же результат

BFS выбирает путь длины 2 (s→A→t), игнорируя длинный путь через A→B. DFS может «увязнуть» в длинных путях через узкое ребро A→B (пропускная способность 1).

Вывод асимптотики $O(V \cdot E^2)$

Шаг 1: Сколько раз умирает одно ребро?

Между двумя смертями ребра (u,v): d(u) растёт минимум на 2.
$d(u) \in [1, V-1] \implies$ одно ребро умирает не более $(V-1)/2 = O(V)$ раз.

Шаг 2: Сколько итераций всего?

Каждая итерация убивает 1 ребро. Рёбер в остаточной сети 2E.
Всего итераций: $2E \times O(V) = O(V \cdot E)$.

Шаг 3: Стоимость одной итерации

BFS по остаточной сети: $O(E)$.
Проход по пути + обновление: $O(V) \subseteq O(E)$.

Шаг 4: Итого

$O(V \cdot E)$ итераций $\times$ $O(E)$ за итерацию $= O(V \cdot E^2)$

Оптимизация: масштабирование потока

Идея: не проталкиваем поток по 1, а начинаем с крупных порций.

Пусть C — максимальная пропускная способность. Начинаем с $k = \lfloor \log_2 C \rfloor$.
На каждом уровне k проталкиваем потоки $\geq 2^k$. Когда больше не можем — уменьшаем k на 1.

	Форд-Фалкерсон (DFS)	Эдмондс-Карп (BFS)	С масштабированием
Сложность	$O(E \cdot \lvert f^* \rvert)$	$O(V \cdot E^2)$	$O(E^2 \cdot \log C)$
Зависит от весов?	Да	Нет	Слабо (log C)
Иррац. веса	Может зависнуть	Завершается	Завершается

03. Минимальный разрез

Узкое место сети

Зачем нужен минимальный разрез?

Задача

Противник хочет полностью заблокировать трубопровод, перекрыв минимальное число труб (с минимальной суммарной пропускной способностью). Какие трубы — «узкое место» сети?

Разрез (S, T) — разбиение вершин на два множества: S (содержит s) и T (содержит t).

Пропускная способность разреза c(S, T) = сумма пропускных способностей рёбер из S в T.

Минимальный разрез — разрез с минимальной пропускной способностью.

Ключевые леммы:

1. Поток через любой разрез = величина потока $|f|$
2. Для любых потока f и разреза (S,T): $|f| \le c(S,T)$
3. Если $|f| = c(S,T)$, то f — максимальный, (S,T) — минимальный

Теорема: max-flow = min-cut

Теорема: Величина максимального потока равна пропускной способности минимального разреза.

Доказательство:

Из леммы 2 знаем: $|f| \le c(S,T)$ для любых f и (S,T).

Значит $\max |f| \le \min c(S,T)$. Осталось показать равенство.

Когда алгоритм Форда-Фалкерсона остановился, BFS не нашёл пути из s в t в остаточной сети.

Построим разрез: S = {вершины, достижимые из s в остаточной сети}, $T = V \setminus S$.

Для любого ребра (u,v) где $u \in S$, $v \in T$:

- остаточная пропускная способность = 0 (иначе v было бы достижимо → $v \in S$)
- значит $f(u,v) = c(u,v)$ — ребро полностью насыщено

Итого: $|f| = f(S,T) = c(S,T)$, значит этот поток максимален, а разрез минимален. ∎

04. Потоки и паросочетания

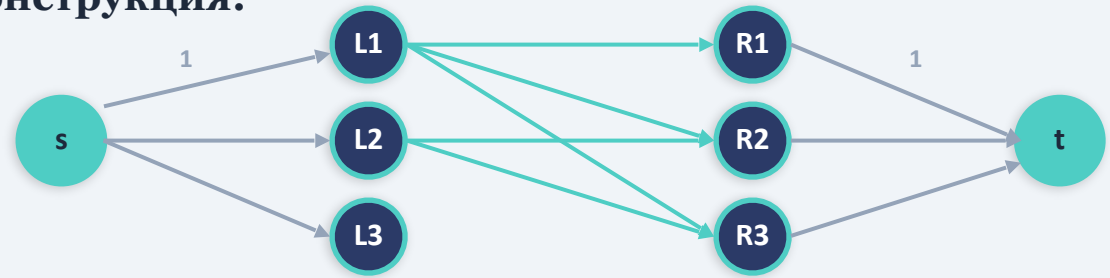
Сведение одной задачи к другой

Зачем сводить паросочетание к потоку?

Идея

Потоки — универсальный инструмент. Множество комбинаторных задач (паросочетания, покрытия, разбиения) сводятся к задаче о максимальном потоке. Один алгоритм — десятки применений.

Конструкция:



1. Добавляем источник s и сток t
2. s → каждая вершина левой доли (пропускная способность 1)
3. Каждая вершина правой доли → t (пропускная способность 1)
4. Рёбра двудольного графа: пропускная способность 1, L → R

Максимальный поток = максимальное паросочетание!

Итоги

Задача	Алгоритм	Сложность
Паросочетание	Алгоритм Куна	$O(V \cdot E)$
Максимальный поток	Форд-Фалкерсон (DFS)	$O(E \cdot \lvert f^* \rvert)$
Максимальный поток	Эдмондс-Карп (BFS)	$O(V \cdot E^2)$
Максимальный поток	С масштабированием	$O(E^2 \cdot \log C)$
Max-flow = Min-cut	Теорема	—

Потоки — универсальный инструмент для решения комбинаторных задач на графах